Cybersecurity Lab 1 · ELTE · Hamdan Al-Jabri · October 2026

Post-quantum key exchange: Part 1 in three pages

How fast are the new quantum-safe methods, and do they work correctly? (experiments E1 and E4)

69 of 69
correctness checks passed
on both computers, incl. NIST's official test vectors

2.2× / 3.5×
ML-KEM is faster than today's X25519
for one whole key exchange (Mac / Intel laptop)

1.6× / 1.7×
cost of the "hybrid" vs X25519
hybrid = X25519 and ML-KEM together, as browsers use

43–841×
cost of the cautious methods
HQC, FrodoKEM, Classic McEliece vs X25519

25–16,000×
more data sent than X25519
ML-KEM-768: 2,272 bytes instead of 64

up to 1,400×
faster with optimised code
the right CPU instructions matter a lot

1 The problem in one paragraph

Before two computers can talk securely they agree on a secret key. Today this is done with **X25519**, which a large quantum computer could break. Recorded traffic could then be decrypted later. New "post-quantum" methods are designed to resist quantum computers. This project measures **what they cost** (time, memory, bytes sent) and **whether the implementations are correct**.

2 Which methods we compare, and why these four

Post-quantum methods are built on two kinds of hard maths problems: **lattices** and **error-correcting codes**. Each kind comes in a **structured** version (extra patterns make it fast and compact, but give attackers more to work with) and an **unstructured** version (slower and bigger, but considered safer). We took one method from each of the four boxes:

	Lattice problems	**Code problems**
Structured (fast, compact)	**ML-KEM** the official US standard (FIPS 203, 2024)	**HQC** chosen by NIST in 2025 as the backup
Unstructured (cautious)	**FrodoKEM** ISO standard (2026), recommended in Germany	**Classic McEliece** ISO standard (2026), oldest unbroken design (1978)

This shows two things at once: lattices vs. codes, and **what the extra caution costs**. Each method is tested at three strength levels (NIST levels 1, 3 and 5, as strong as AES-128, -192 and -256), giving 12 versions. They are compared with **X25519** (today's method) and the **hybrid X25519MLKEM768** (both together, used by browsers today).

3 What is a KEM?

All four methods are **KEMs** (key-encapsulation mechanisms). A KEM has three steps (Figure 1):

1. **KeyGen**: the client makes a key pair and sends the public part.
2. **Encaps**: the server picks a random secret and locks it with that public key. It sends the locked box (the "ciphertext").
3. **Decaps**: the client opens the box with its private key.

Now both sides hold the same 32-byte secret. Think of the client sending an open padlock that only it can unlock.

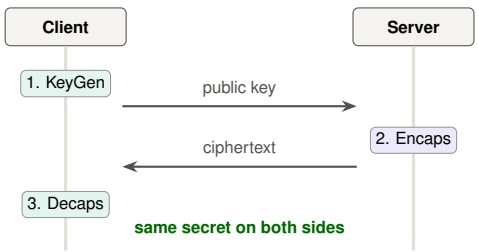


Figure 1. One key exchange with a KEM.

How X25519 fits. X25519 is not a KEM; both sides mix their own secret with the other's public value. It still has the same three-step shape: client key pair = KeyGen; server key pair + mixing = Encaps; client mixing = Decaps. So we measure it in exactly these roles (**X25519-ECDH**, as TLS uses it). We also measured the official "KEM-wrapped" version (**X25519-DHKEM**, RFC 9180). It adds one extra hashing step and costs 10–14% more, so plain ECDH is the fair baseline.

The hybrid sends both public parts together (1,184 + 32 = 1,216 bytes) and combines both secrets. It stays safe if either part is broken.

4 How we measured

- **Two computers, same software:** an **Apple M4** MacBook Air (ARM) and an **Intel i7-1185G7** laptop (x86). Both run Linux, the M4 in Docker and the Intel in WSL2, with the same compiler and libraries (liboqs 0.16.0, OpenSSL 3.5.8). Only the processor differs.
- **Three versions of the code: portable** (plain C that runs anywhere), **optimised** (uses special CPU instructions that process many numbers at once: AVX2 on Intel, NEON on ARM) and **OpenSSL** (the library real websites use, for X25519, ML-KEM and the hybrid). Optimised code exists for ML-KEM on both computers, and for FrodoKEM and McEliece only on Intel. HQC has none.
- **Timing:** every step is run 1,000 times and each run is timed. We report the **median** (the middle value, so a rare slow run does not distort it) and the **spread** of the middle half of the runs.
- **Fair conditions:** a 200 ms warm-up before timing (the M4 needs about 90 ms to reach full speed), a 30-second pause between methods (the fanless Air must cool down), and a measurement of the processor's actual speed. Results can then also be given in **CPU cycles**, the unit used in published papers.
- **McEliece key generation is timed 100 times, not 1,000.** One run takes 40–500 ms, so 1,000 runs would add hours. Its time also varies a lot: it picks a random secret and tries to build the public key, which works only about 29% of the time. Otherwise it starts again. The number of retries is luck, so one run can take six times as long as the next. 20 samples gave an unstable median; 100 give a stable one.

5 E4: is the code correct?

Yes: all 69 checks pass on both computers (1,332 test cases each). We checked:

- **answers against known correct results** for all 12 methods, in both the portable and the optimised version;
- **NIST's official FIPS 203 test vectors** for ML-KEM (360 of 360);
- **random key exchanges** that must give the same secret on both sides;
- that **keys made by one version work with the other** (portable with optimised);
- that **ML-KEM in liboqs and in OpenSSL** work with each other.

So the fast optimised code gives exactly the same results as the plain code, and the speed comparison is fair. One small gap: NIST's input-validation tests are not reachable through liboqs and were skipped.

6 E1: what one key exchange costs

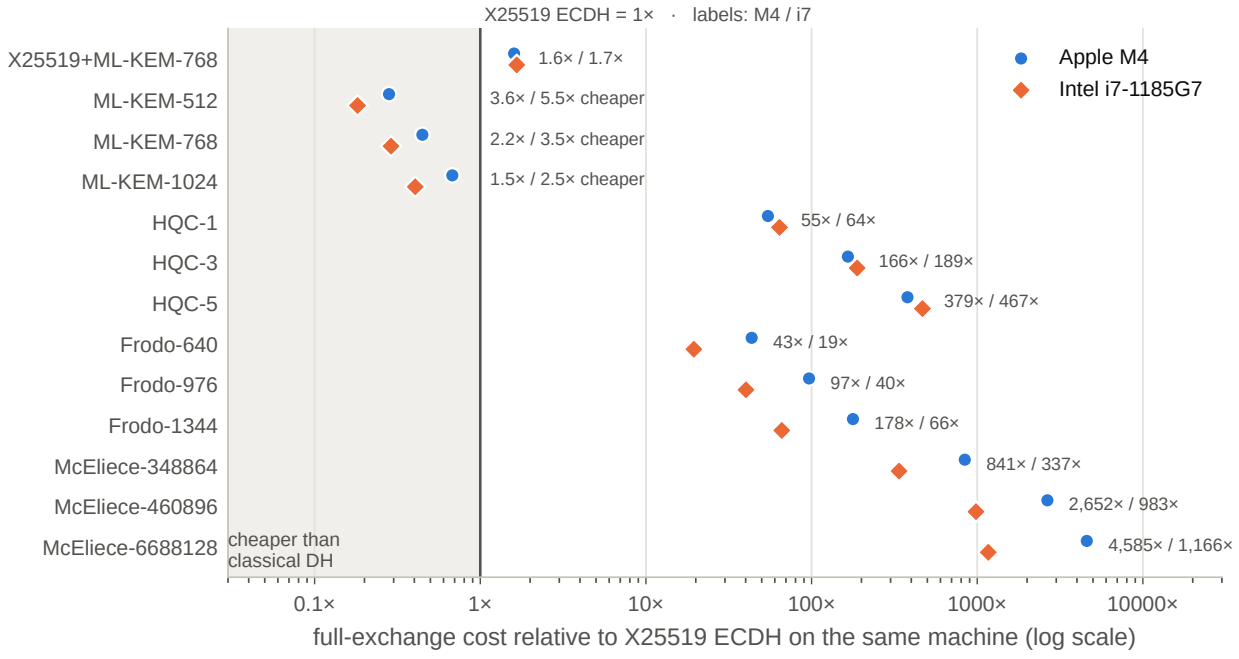


Figure 2. Cost of one whole key exchange (KeyGen + Encaps + Decaps) compared with X25519 on the same computer. Left of the "1×" line (grey area) is cheaper than X25519. Blue dots = Apple M4, orange = Intel i7; labels give M4 / Intel.

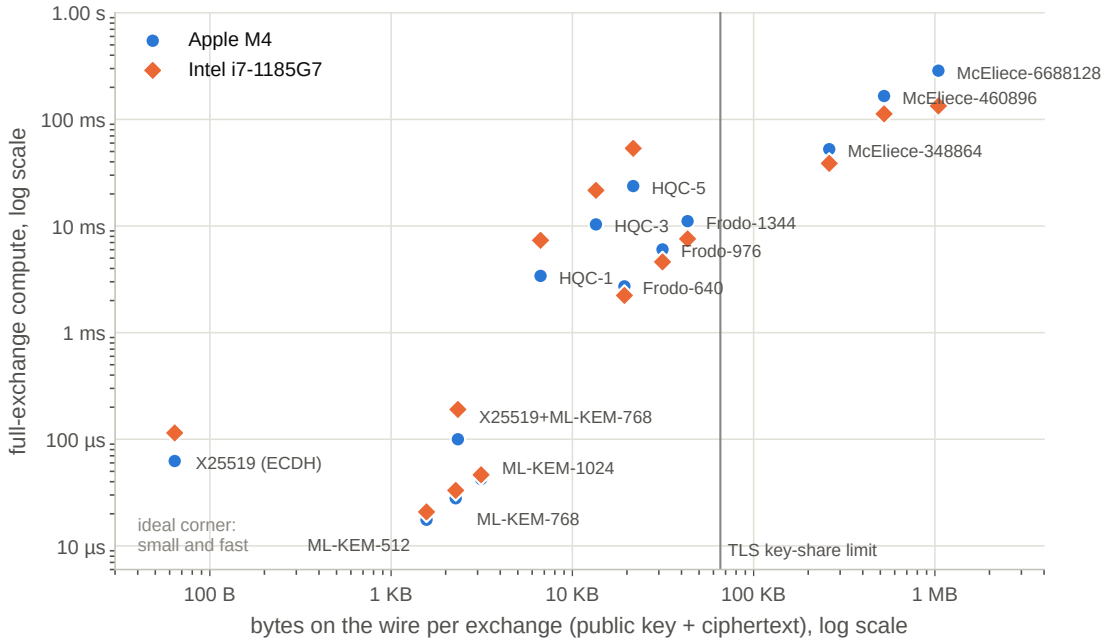


Figure 3. Time vs. bytes sent per key exchange. Bottom-left is best (fast and small). Methods right of the grey line do not fit in one TLS message.

7 What we found

1. **ML-KEM is faster than today's X25519** at every strength level: 2.2× (M4) and 3.5× (Intel) at level 3, and still faster at level 5. Its cost is **size**, not time: it sends 25–49× more bytes, about 1.5–3 KB. The reason: ML-KEM works on many small 12-bit numbers that a CPU handles in batches, while X25519 needs thousands of slow steps on 255-bit numbers.
2. **The hybrid costs 1.6–1.7× X25519**, almost exactly X25519 plus ML-KEM added together, so combining them costs nothing extra.
3. **The cautious methods are much more expensive, in both time and size.** At level 1: HQC 55–64× the time of X25519, FrodoKEM 19–43×, McEliece 337–841×. They send 4–14× more bytes than ML-KEM; McEliece's public key (0.26–1 MB) is too big for TLS. Moving from level 1 to 5 multiplies the cost by about 2.3 for ML-KEM and by 3.4–7 for the others.
4. **The code version matters as much as the method.** Optimised Intel code makes ML-KEM and FrodoKEM about 3× faster and McEliece's Decaps 510–1,400× faster. On the M4, only ML-KEM has optimised code (1.4–1.7× faster). So the M4 is 2.2× faster than the Intel laptop on plain code but slower for FrodoKEM and McEliece. liboqs has no optimised HQC; HQC's designers report code 33–49× faster than what we measured.
5. **Memory:** ML-KEM needs only 10–21 KB of working memory; HQC and FrodoKEM 60–200 KB; McEliece up to 1.5 MB, too much for small devices.
6. **Processor speed changes during the test:** the Intel CPU slows from 3.9 to 3.2 GHz while running optimised code; the M4 stayed between 4.0 and 4.44 GHz.
7. **Our numbers agree with published ones** for HQC, McEliece Encaps/Decaps and all key sizes. ML-KEM and FrodoKEM are faster than the 2021 figures (newer code). **One open question:** McEliece-348864 key generation takes 2.6× the published time.

Limits: both computers run Linux inside a virtual machine; one processor of each kind; the Intel time conversion varied 2.4% between sessions (CPU-cycle results are not affected). **Next (Part 2):** real TLS connections under network delay and loss (E2), and tests for secret-dependent timing leaks (E3). All details and tables are in the full Part 1 report; code and data at `github.com/hsh1994-ux/pq-kem-study`. Prepared with help from Claude (Anthropic); the design and interpretation are the author's.